

By default, the **sort** command sorts in byte order, which is the order in which the characters appear in the ASCII text table. Notice that this looks like alphabetical order, but it is not, as all capital letters are shown before lowercase letters. So *Zoo* would be listed before *apple*. In some cases, that is not convenient because the content that needs sorting may be numeric or in another format. The **sort** command offers different options to help sorting these specific types of data. Type, for instance, **cut -f 3 -d : /etc/passwd | sort -n** to sort the third field of the /etc/passwd file in numeric order. It can be useful also to sort in reverse order; if you use the command **du -h | sort -rn**, you get a list of files sorted with the biggest file in that directory listed first.

You can also use the **sort** command and specify which column you want to sort. To do this, use **sort -k3 -t : /etc/passwd**, for instance, which uses the field separator : to sort the third column of the /etc/passwd file. Add **-n** to the command to sort in a numeric order, and not in an alphabetic order.

Another example is shown in [Example 4-2](#), where the output of the **ps aux** command is sorted. This command gives an overview of processes running on a Linux system. The fourth column indicates memory usage, and by applying a numeric sort to the output of the command, you can see that the processes are sorted by memory usage, such that the process that consumes the most memory is listed last.

Example 4-2 Using **ps aux** to Find the Busiest Processes on a Linux Server

[Click here to view code image](#)

```
[root@localhost ~]# ps aux | sort -k 4 -n
root      897   0.3   1.1 348584 42200 ?        Ssl  08:12   0:00
    /usr/bin/python3 -s /usr/sbin/firewalld --nofork --nopid
student   2657   1.0   1.1 2936188 45200 ?        Ssl  08:14   0:00
    /usr/bin/gjs /usr/share/org.gnome.Characters/org.gnome.Characters.
    BackgroundService
student   2465   0.3   1.3 143976 52644 ?        S    08:14   0:00
    /usr/bin/Xwayland :0 -rootless -noreset -accessx -core -auth /
    run/user/1000/.mutter-Xwaylandauth.0SRUV1 -listenfd 4 -listenfd 5
    -displayfd 6 -initfd 7
student   2660   1.9   1.4 780200 53412 ?        Ssl  08:14   0:00
    /usr/libexec/gnome-terminal-server
root      2480   2.1   1.6 379000 61568 ?        Ssl  08:14   0:00
    /usr/bin/python3 /usr/libexec/rhsm-service
student   2368   0.9   1.6 1057048 61096 ?        Sl   08:14   0:00
    /usr/libexec/evolution-data-server/evolution-alarm-notify
root      1536   0.6   1.8 555908 69916 ?        Ssl  08:12   0:00
    /usr/libexec/packagekitd
student   2518   0.6   1.8 789408 70336 ?        Ssl  08:14   0:00
    /usr/libexec/gsd-xsettings
student   2540   0.5   1.8 641720 68828 ?        Sl   08:14   0:00
    /usr/libexec/ibus-x11 --kill-daemon
student   2381   4.7   1.9 1393476 74756 ?        Sl   08:14   0:00
    /usr/bin/gnome-software --gapplication-service
student   2000  16.0   7.8 3926096 295276 ?       Ssl  08:14   0:03
    /usr/bin/gnome-shell
```

Counting Lines, Words, and Characters with wc

When working with text files, you sometimes get a large amount of output. Before deciding which approach to handling the large amount of output works best in a specific case, you might want to have an idea about the amount of text you are dealing with. In that case, the **wc** command is useful. In its output, this command gives three different results: the number of lines, the number of words, and the number of characters.

Consider, for example, the **ps aux** command. When executed as root, this command gives a list of all processes running on a server. One solution to count how many processes there are exactly is to pipe the output of **ps aux** through **wc**, as in **ps aux | wc**. You can see the result of the command in [Example 4-3](#), which shows that the total number of lines is 90 and that there are 1,045 words and 7,583 characters in the command output.

Example 4-3 Counting the Number of Lines, Words, and Characters with **wc**

[Click here to view code image](#)

```
[root@localhost ~]# ps aux | wc
  90      1045      7583
```

A Primer to Using Regular Expressions

Working with text files is an important skill for a Linux administrator. You must know not only how to create and modify existing text files, but also how to find the text file that contains specific text.

It will be clear sometimes which specific text you are looking for. Other times, it might not. For example, are you looking for *color* or *colour*? Both spellings might give a match. This is just one example of why using flexible patterns while looking for text can prove useful. In Linux these flexible patterns are known as *regular expressions*, often also referred to as *regex*.

To understand regular expressions a bit better, let's take a look at a text file example, shown in [Example 4-4](#). This file contains the last six lines from the `/etc/passwd` file. (This file is used for storing Linux accounts; see [Chapter 6](#), “[User and Group Management](#),” for more details.)

Example 4-4 Sample Lines from `/etc/passwd`

[Click here to view code image](#)

```
[root@localhost ~]# tail -n 6 /etc/passwd
anna:x:1000:1000:./home/anna:/bin/bash
rihanna:x:1001:1001:./home/rihanna:/bin/bash
annabel:x:1002:1002:./home/annabel:/bin/bash
anand:x:1003:1003:./home/anand:/bin/bash
joanna:x:1004:1004:./home/joanna:/bin/bash
joana:x:1005:1005:./home/joana:/bin/bash
```

Now suppose that you are looking for the user `anna`. In that case, you could use the general regular expression parser **grep** to look for that specific string in the file `/etc/passwd` by using the command **grep anna /etc/passwd**. [Example 4-5](#) shows the results of that command, and as you can see, way too many results are shown.

Example 4-5 Example of Why You Need to Know About Regular Expressions

[Click here to view code image](#)

```
[root@localhost ~]# grep anna /etc/passwd
anna:x:1000:1000:./home/anna:/bin/bash
rihanna:x:1001:1001:./home/rihanna:/bin/bash
annabel:x:1002:1002:./home/annabel:/bin/bash
joanna:x:1004:1004:./home/joanna:/bin/bash
```



A regular expression is a search pattern that allows you to look for specific text in an advanced and flexible way.

Using Line Anchors

In [Example 4-5](#), suppose that you wanted to specify that you are looking for lines that start with the text *anna*. The type of regular expression that specifies where in a line of output the result is expected is known as a *line anchor*.

To show only lines that start with the text you are looking for, you can use the regular expression `^` (in this case, to indicate that you are looking only for lines where *anna* is at the beginning of the line; see [Example 4-6](#)).

Example 4-6 Looking for Lines Starting with a Specific Pattern

[Click here to view code image](#)

```
[root@localhost ~]# grep ^anna /etc/passwd
anna:x:1000:1000:./home/anna:/bin/bash
annabel:x:1002:1002:./home/annabel:/bin/bash
```

Another regular expression that relates to the position of specific text in a specific line is `$`, which states that the line ends with some text. For instance, the command **grep ash\$ /etc/passwd** shows all lines in the `/etc/passwd` file that end with the text *ash*. This command shows all accounts that have a shell and are able to log in (see [Chapter 6](#) for more details).

Using Escaping in Regular Expressions

Although not mandatory, when you’re using regular expressions, it is a good idea to use *escaping* to prevent regular expressions from being interpreted by the shell. When a command line is entered, the Bash shell parses the command line, looking for any special characters like *, \$, and ?. It will next interpret these characters. The point is that regular expressions use some of these characters as well, and to make sure the Bash shell doesn’t interpret them, you should use escaping.

In many cases, it is not really necessary to use escaping; in some cases, the regular expression fails without escaping. To prevent this from ever happening, it is a good idea to put the regular expression between quotes. So, instead of typing `grep ^anna /etc/passwd`, it is better to use `grep '^anna' /etc/passwd`, even if in this case both examples work.

Using Wildcards and Multipliers

In some cases, you might know which text you are looking for, but you might not know how the specific text is written. Or you might just want to use one regular expression to match different patterns. In those cases, wildcards and multipliers come in handy.

To start with, there is the dot (.) regular expression. This is used as a *wildcard* character to look for one specific character. So, the regular expression `r.t` would match the strings *rat*, *rot*, and *rut*.

In some cases, you might want to be more specific about the characters you are looking for. If that is the case, you can specify a range of characters that you are looking for. For instance, the regular expression `r[ao]t` matches the strings *rat*, *rot*, and *rut* but it wouldn’t match *rit* and *ret*.

Another useful regular expression is the *multiplier* *. This matches zero or more of the previous character. That does not seem to be very useful, but indeed it is, as you will see in the examples at the end of this section.

If you know exactly how many of the previous character you are looking for, you can specify a number also, as in `re{2}d`, which would match *reed*, but not *red*. The last regular expression that is useful to know about is ?, which matches zero or one of the previous character. [Table 4-3](#) provides an overview of the most important regular expressions.

Using Extended Regular Expressions

What makes regular expressions sometimes a bit hard to understand is the fact that there are different sets of regular expressions. The base regular expressions as discussed so far are supported by tools like **grep**. There is also a set of extended regular expressions, which is not supported by default. When used with **grep**, you’ll have to add the -E option to indicate it is an extended regular expression. The + can be used to indicate that a character should occur one or more times, and the ? is used to indicate that a character should occur zero or one times. When used in **grep**, don’t forget to use **grep -E** to ensure that these are interpreted as extended regular expressions!



Table 4-3 Most Significant Regular Expressions

Regular Expression	Use
^text	Matches line that starts with specified text.
text\$	Matches line that ends with specified text.
.	Wildcard. (Matches any single character.)
[abc]	Matches <i>a</i> , <i>b</i> , or <i>c</i> .
?	Extended regular expression that matches zero or one of the preceding character.
+	Extended regular expression that matches one or more of the preceding character.
*	Matches zero to an infinite number of the previous character.
\{2\}	Matches exactly two of the previous character.

Regular Expression	Use
\{1,3\}	Matches a minimum of one and a maximum of three of the previous character.
colou?r	Matches zero or one of the previous character. This makes the previous character optional, which in this example would match both <i>color</i> and <i>colour</i> .
(...)	Used to group multiple characters so that the regular expression can be applied to the group.

Let’s take a look at an example of a regular expression that comes from the man page `semanage-fcontext` and relates to managing SELinux (see [Chapter 22](#), “[Managing SELinux](#)”). The sample line contains the following regular expression:

```
"/web(/ . *)?"
```

In this regular expression, the text `/web` is referred to. This text string can be followed by the regular expression `(/.*)?`. To understand the regular expression, start with the `?`, which refers to the part between braces and indicates that the part between braces may occur zero times or one time. Within the braces, the pattern starts with a slash, which is just a slash, followed by zero or more characters. So this means that just the directory name gives a match, but also the directory name followed by just a slash, or a slash that is followed by a filename.

What makes regular expressions difficult is that there is not just one set of regular expressions; there are also extended regular expressions. And to make the concept more complex, the extended regular expressions need specific commands. The well-known command **grep** (covered next) by default deals with base regular expressions. If you want to use extended regular expressions, you need **grep -E** or **egrep**.

Two common extended regular expressions are `+` and `?`. The `+` will look for a pattern where the preceding character occurs one or more times, and the `?` looks for a pattern where the preceding character does not occur or occurs one time. Use the following procedure to find out how these extended regular expressions can be confusing:

Step 1. Create a text file with the name **regex.txt** and the following contents:

```
bat
boot
boat
bt
```

Step 2. Use **grep 'b.*t' regex.txt** to see any line that starts with a *b* and ends with a *t*.

Step 3. Use **grep 'b.+t' regex.txt**. You might expect to see only lines that have at least three characters, but you don’t, because you are using an extended regular expression, and without using any additional options, **grep** doesn’t recognize the extended regular expression.

Step 4. Use **grep -E 'b.+t' regex.txt**. Now you see that the extended regular expression does work as expected.

Using grep to Analyze Text

The ultimate utility to work with regular expressions is **grep**, which stands for “general regular expression parser.” Quite a few examples that you have seen already were based on the **grep** command. The **grep** command has a couple of useful options to make it even more efficient. [Table 4-4](#) describes some of the most useful options.



Table 4-4 Most Useful **grep** Options

Option	Use
-i	Matches upper- and lowercase letters (i.e., not case sensitive).
-v	Shows only lines that do <i>not</i> contain the regular expression.

[Click here to view code image](#)

```
sed -i s/old-text/new-text/g ~/myfile
```

In this example, the **sed** utility is used to search for the text *old-text* in *~/myfile* and replace all occurrences with the text *new-text*. Notice that the default **sed** behavior is to write the output to STDOUT, but the option **-i** will write the result directly to the file. Make sure that you know what you are doing before using this command, because it might be difficult to revert file modifications that are applied in this way.

You’ll like the following example if you’ve ever had a utility containing a specific line in a file that was erroneous:

```
sed -i -e '2d' ~/myfile
```

With this command, you can delete a line based on a specific line number. You can also make more complicated references to line numbers. Use, for instance, **sed -i -e '2d;20,25d' ~/myfile** to delete lines 2 and 20 through 25 in the file *~/myfile*.

Tip

Do not focus on **awk** and **sed** too much. These are amazing utilities, but many of the things that can be accomplished using them can be done using other tools as well. The **awk** and **sed** tools are very rich, and you can easily get lost in them if you are trying to dig too deep.

Summary

In this chapter, you learned how to work with text files. You acquired some important skills like searching text files with **grep** and displaying text files or part of them with different utilities. You also learned how regular expressions can be used to make the search results more specific. Finally, you learned about the very sophisticated utilities **awk** and **sed**, which allow you to perform more advanced operations on text files.

Exam Preparation Tasks

As mentioned in the section “[How to Use This Book](#)” in the Introduction, you have several choices for exam preparation: the end-of-chapter labs; the memory tables in [Appendix C](#); [Chapter 27](#); “[Final Preparation](#)”; and the practice exams.

Review All Key Topics

Review the most important topics in the chapter, noted with the Key Topic icon in the margin of the page. [Table 4-5](#) lists a reference for these key topics and the page number on which each is found.



Table 4-5 Key Topics for Chapter 4

Key Topic Element	Description	Page
Table 4-2	Essential Tools for Managing Text File Contents	84
Paragraph	Definition of a regular expression	90
Table 4-3	Most Significant Regular Expressions	92
Table 4-4	Most Useful grep Options	94